

KAFKA HANDBOOK

Kafka From Zero To Hero Notes

1. WHAT IS KAFKA?

Apache Kafka is a distributed event streaming platform used for building real-time data pipelines and streaming applications.

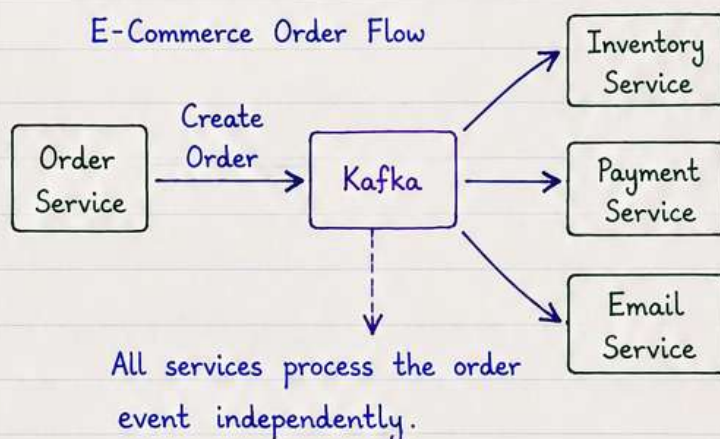
2. WHY KAFKA?

- ✓ High Throughput
- ✓ Scalable & Distributed
- ✓ Fault Tolerant
- ✓ Real-time Data Processing
- ✓ Durable & Reliable
- ✓ Decouples Systems

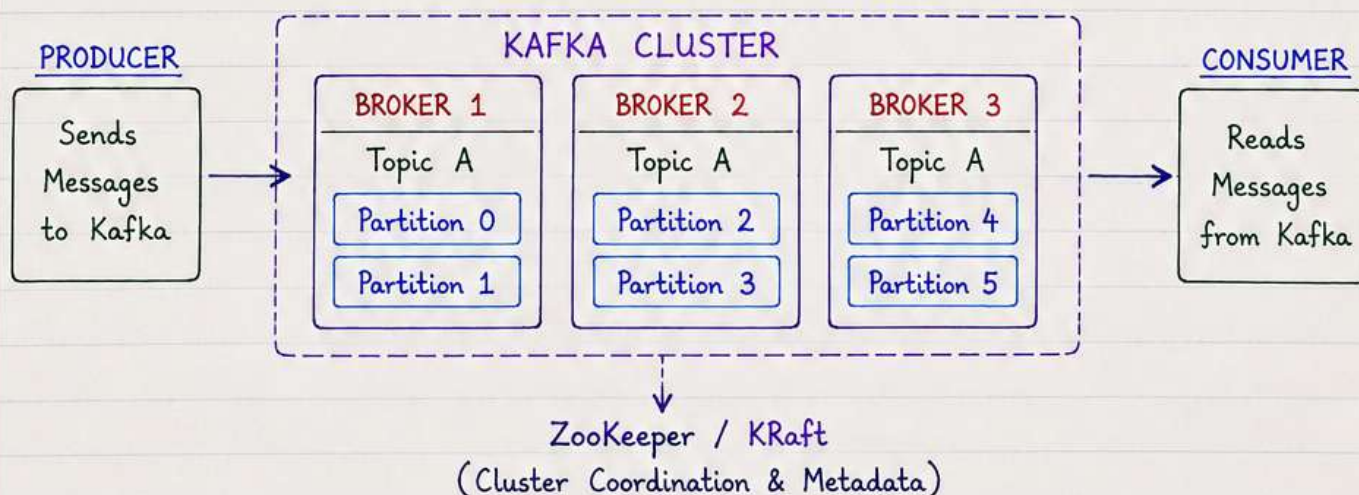
3. PROBLEMS WITH TRADITIONAL COMMUNICATION

- ✗ Systems are tightly coupled.
- ✗ If consumer is down, producer fails.
- ✗ No buffer for high volume of data.
- ✗ Hard to scale.
- ✗ No real-time data processing.

4. REAL-WORLD EXAMPLE



5. CORE COMPONENTS OVERVIEW



6. COMPANIES USING KAFKA

Netflix

Uber

LinkedIn

Swiggy

Amazon

Zomato

NOTE

Kafka is not just a message queue, it is a distributed event streaming platform. It stores data, processes data, and delivers data in real-time.

2. KAFKA ARCHITECTURE

Kafka follows a distributed, partitioned and replicated architecture to achieve high throughput, scalability and fault tolerance.

CORE COMPONENTS

① BROKER

- A Kafka server that stores data and handles requests.

② TOPIC

- A logical channel to which producers publish messages.

③ PARTITION

- A topic is split into multiple partitions. Each partition is ordered and immutable.

④ OFFSET

- A unique number assigned to each message within a partition.

⑤ REPLICATION

- Partitions are replicated across multiple brokers for reliability.

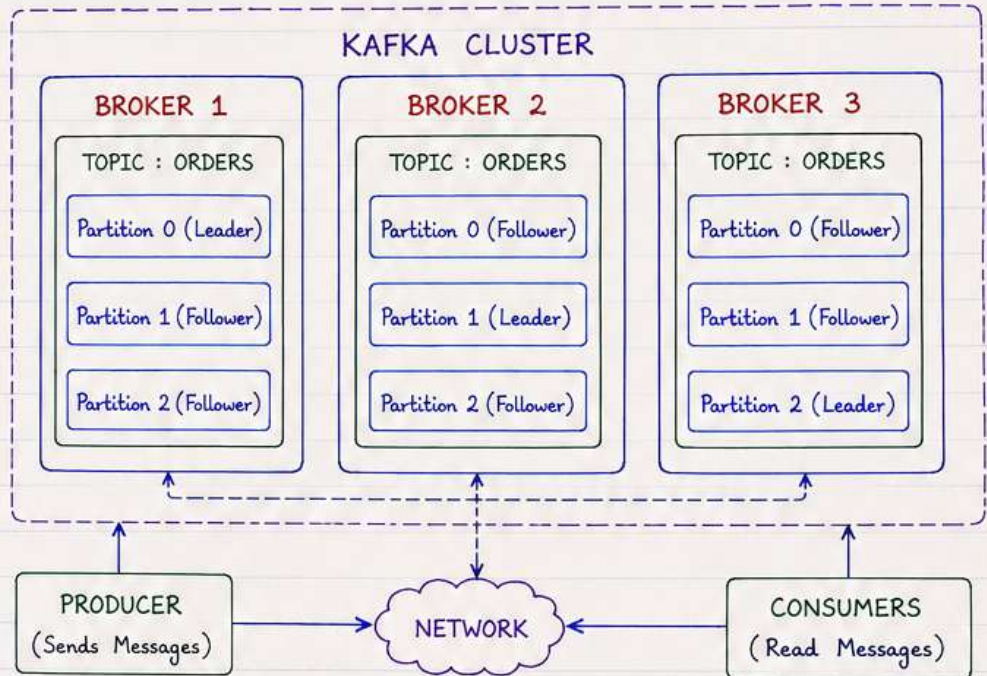
⑥ ZOOKEEPER (OLD)

- Used for broker coordination and metadata management. (In new versions replaced by KRaft)

⑦ KRAFT (NEW)

- Kafka Raft metadata mode. No need of ZooKeeper. Built on Raft consensus.

ARCHITECTURE DIAGRAM



HOW IT WORKS ?

- Producer publishes messages to a Topic.
- Topic is divided into multiple Partitions.
- Each Partition has a Leader and Followers (replicas).
- Consumers read messages from Partitions.
- Offsets keep track of messages.
- If Leader fails, a Follower becomes new Leader.

EXAMPLE

Topic : Orders

- Partition 0 → Leader (Broker 1)
Replicas → Broker 2, Broker 3
- Partition 1 → Leader (Broker 2)
Replicas → Broker 1, Broker 3
- Partition 2 → Leader (Broker 3)
Replicas → Broker 1, Broker 2

OLD vs NEW (ZOOKEEPER vs KRAFT)

KEY BENEFITS

- ✓ High Throughput
- ✓ Scalability (Horizontal)
- ✓ Fault Tolerance
- ✓ Durability (Data Replication)
- ✓ Decouples Systems
- ✓ Real-time Data Streaming

FEATURE	ZOOKEEPER (OLD)	KRAFT (NEW)
Metadata Management	Handled by ZooKeeper	Handled by Kafka itself
Extra Dependency	Yes (Needs ZooKeeper)	No (Built-in)
Performance	Slightly slower	Better
Setup	Complex	Simpler
Consensus	External	Raft Consensus (Internal)

NOTE :

- * A Topic can have many Partitions.
- * Each Partition is assigned to one Leader and multiple Followers.

REMEMBER

Partition = Parallelism
Replication = Reliability

3. PRODUCER & CONSUMER

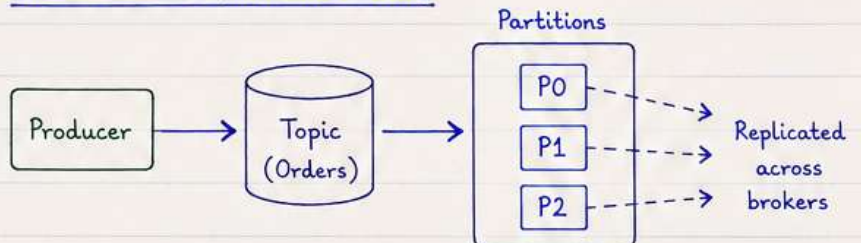
Kafka follows a simple yet powerful model:



3.1 PRODUCER

- ① Producer creates a message.
- ② It sends the message to a Topic.
- ③ Kafka decides the Partition (based on key or round-robin).
- ④ Message is stored in the Partition by the Leader.

MESSAGE FLOW (PRODUCER SIDE)



JAVA PRODUCER EXAMPLE (Spring Boot)

```

@Autowired
private KafkaTemplate<String, String> kafkaTemplate;

public void sendMessage() {
    String topic = "orders-topic";
    String key = "order-123";
    String value = "Order Created";

    // send message
    kafkaTemplate.send(topic, key, value);
}
  
```

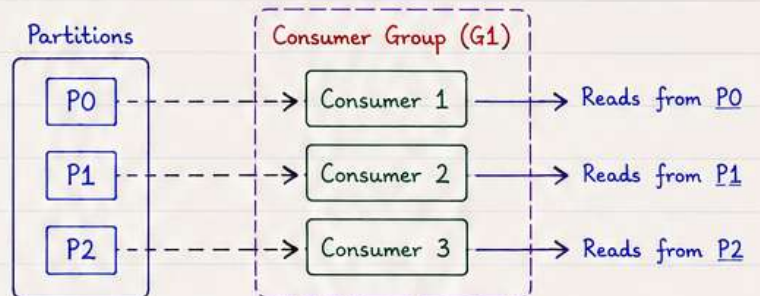
KEY POINTS

- ✓ Key helps in deciding the Partition.
- ✓ Same key always goes to same Partition.
- ✓ If no key → messages are distributed in round-robin manner.

3.2 CONSUMER

- ① Consumer subscribes to one or more Topics.
- ② It belongs to a Consumer Group.
- ③ It pulls (polls) messages from Kafka.
- ④ It processes the message.
- ⑤ It commits the Offset after successful processing.

MESSAGE FLOW (CONSUMER SIDE)



KEY POINTS

- ✓ Partitions are distributed among consumers within the same group.
- ✓ If a consumer is down, others will rebalance and take over its partitions.
- ✓ Consumer Group → enables scaling and fault tolerance.

JAVA CONSUMER EXAMPLE (Spring Boot)

```

@KafkaListener(topics = "orders-topic", groupId = "order-group")
public void listen(String message) {
    System.out.println("Received : " + message);

    // Process message
}
  
```

IMPORTANT TERMS

- | | | | |
|--------------------|--------------------------------|-------------------------|--|
| • Key | : Used to decide Partition. | • Offset | : Unique ID of message in a Partition. |
| • Value | : Actual message (payload). | • Consumer Group | : Group of consumers working together. |
| • Topic | : Logical channel. | • Commit | : Saving the offset after processing. |
| • Partition | : Parallel log within a Topic. | | |

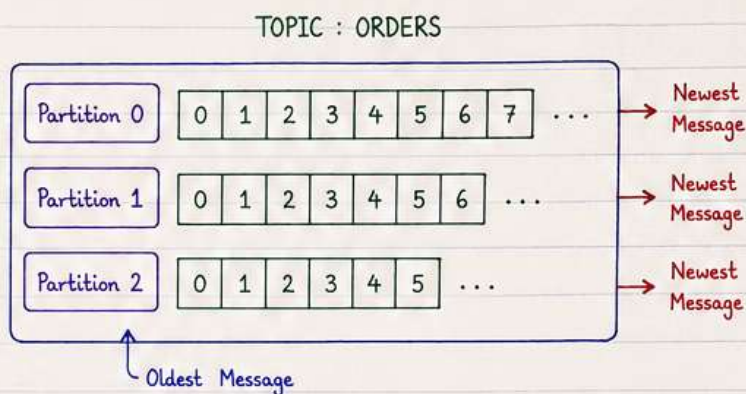
Producer → Writes messages to Kafka.

Consumer → Reads messages from Kafka.

4. TOPICS, PARTITIONS & CONSUMER GROUPS

* **TOPIC** : A topic is a category or feed name to which messages are published.

* **PARTITION** : A topic is split into multiple partitions. Each partition is an ordered, immutable sequence of messages.



WHY PARTITIONS ?

- ✓ Enables parallelism (more consumers can read simultaneously).
- ✓ Increases throughput.
- ✓ Allows horizontal scaling.
- ✓ Maintains order within each partition.
- ✓ Improves fault tolerance.

IMPORTANT POINTS

- Order is guaranteed only within a partition.
- Messages in different partitions may be processed out of order.
- More partitions → more parallelism.
- Too many partitions → overhead.

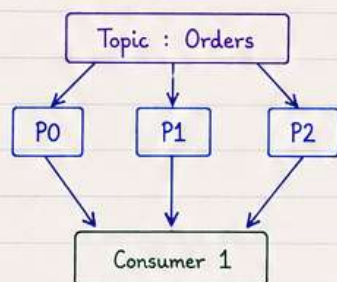
CONSUMER GROUP : A group of consumers that work together to consume messages from a topic.

RULE : In a consumer group, one partition can be consumed by only one consumer at a time.

HOW CONSUMER GROUPS WORK ?

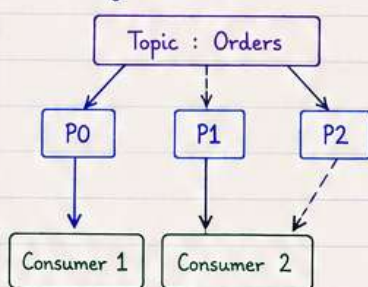
CASE 1 : 1 CONSUMER (G1)

All partitions are read by one consumer.



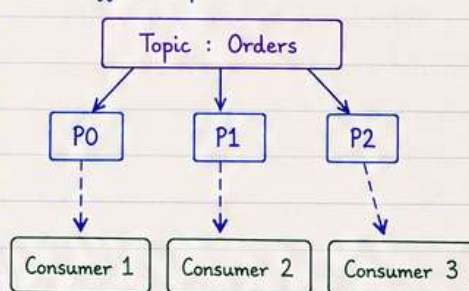
CASE 2 : 2 CONSUMERS (G1)

Partitions are distributed among consumers.



CASE 3 : 3 CONSUMERS (G1)

Each consumer reads from different partition.



REBALANCING

- When a consumer joins or leaves a group, Kafka automatically rebalances partitions.
- Partitions are reassigned to active consumers.
- This ensures high availability and efficient utilization.

EXAMPLE :

Topic has 3 partitions (P0, P1, P2)
Consumer Group (G1)

- If 2 consumers are active → p0 to C1, p1 to C2, p2 to C1
- If 1 consumer leaves → Rebalance happens and remaining consumer takes over partitions.

KEY TAKEAWAYS

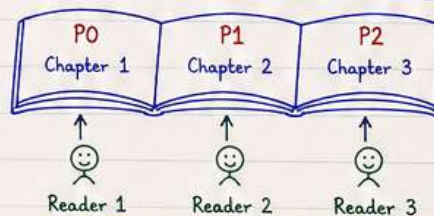
- ✓ More partitions → higher throughput.
- ✓ Order is guaranteed only inside partition.
- ✓ Consumer Group enables parallel and fault-tolerant processing.
- ✓ Rebalancing keeps the system resilient.

ANALOGY



Think of Kafka Topic as a Book.
Partitions are Chapters (P0, P1, P2).
Consumers are Readers.

A reader can read one chapter at a time, and multiple readers can read different chapters together.

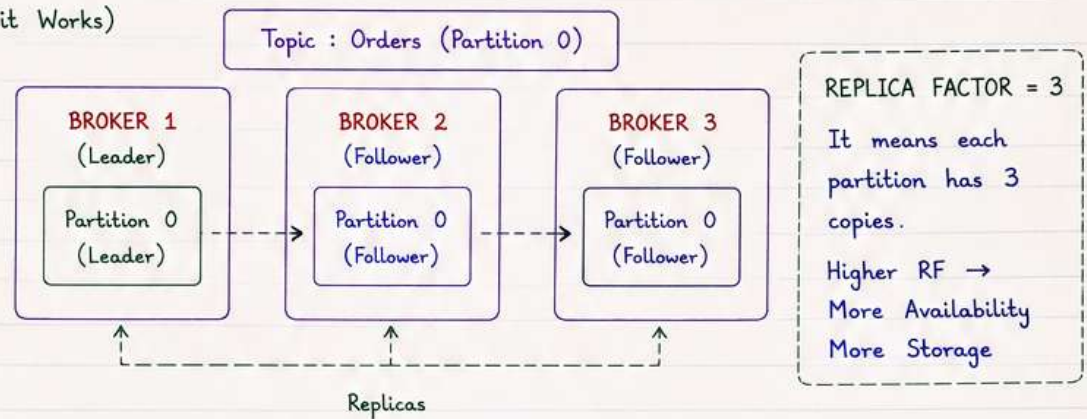


5. REPLICATION, FAULT TOLERANCE & DELIVERY GUARANTEES

Kafka is designed to be highly available and fault tolerant using Replication.

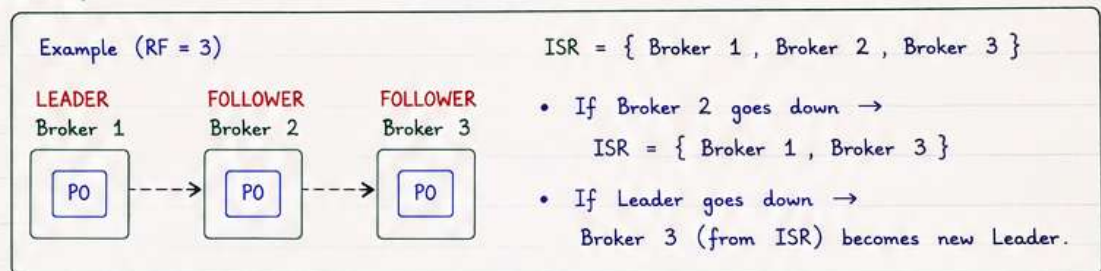
5.1 REPLICATION (How it Works)

- Each partition is replicated across multiple brokers.
- One broker acts as **Leader**.
- Others are **Followers**.
- If Leader fails, one Follower becomes new Leader.

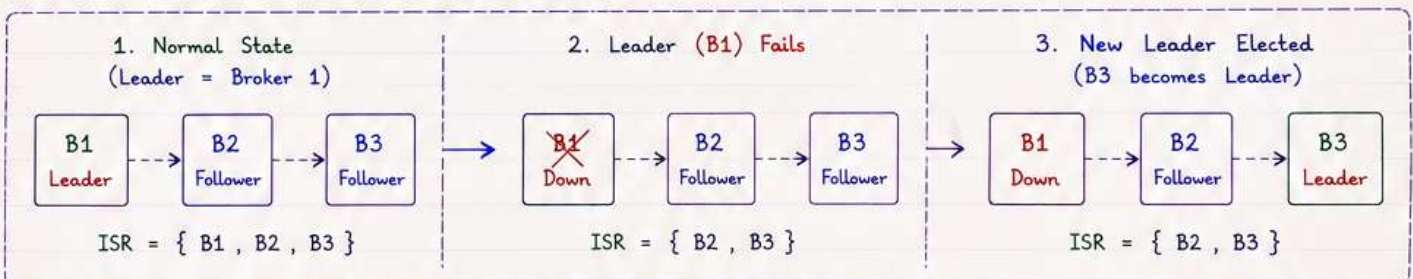


5.2 ISR (IN-SYNC REPLICAS)

- ISR are replicas that are fully caught up with the Leader.
- Only ISR replicas can be new Leader.



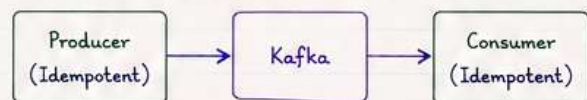
5.3 WHAT HAPPENS WHEN LEADER FAILS ?



5.4 DELIVERY GUARANTEES

GUARANTEE	DESCRIPTION	USE CASE
AT MOST ONCE	Message may be lost but never processed more than once.	Logging, Metrics
AT LEAST ONCE	Message will never be lost but may be processed more than once.	Emails, Notifications, Order Processing
EXACTLY ONCE	Message will never be lost and never be processed more than once.	Payments, Financial Transactions

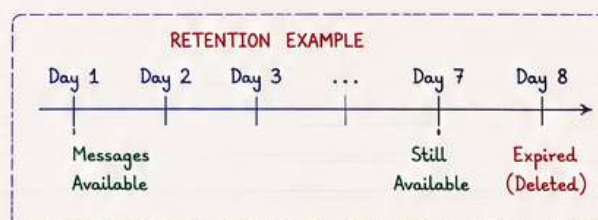
HOW EXACTLY ONCE WORKS (EOS)



- Idempotent Producer → Avoids duplicate messages.
- Transactions → Ensures atomicity (All or Nothing).
- Consumer reads only committed messages.

5.5 RETENTION & MESSAGE LIFETIME

- Kafka stores messages for a configurable amount of time.
- Defined by:
 - `retention.ms` (time based)
 - `retention.bytes` (size based)
- Default: 7 days (for most setups).



WHY RETENTION ?

- ✓ Reprocess data
- ✓ Replay events
- ✓ Audit & Compliance
- ✓ Consumer recovery

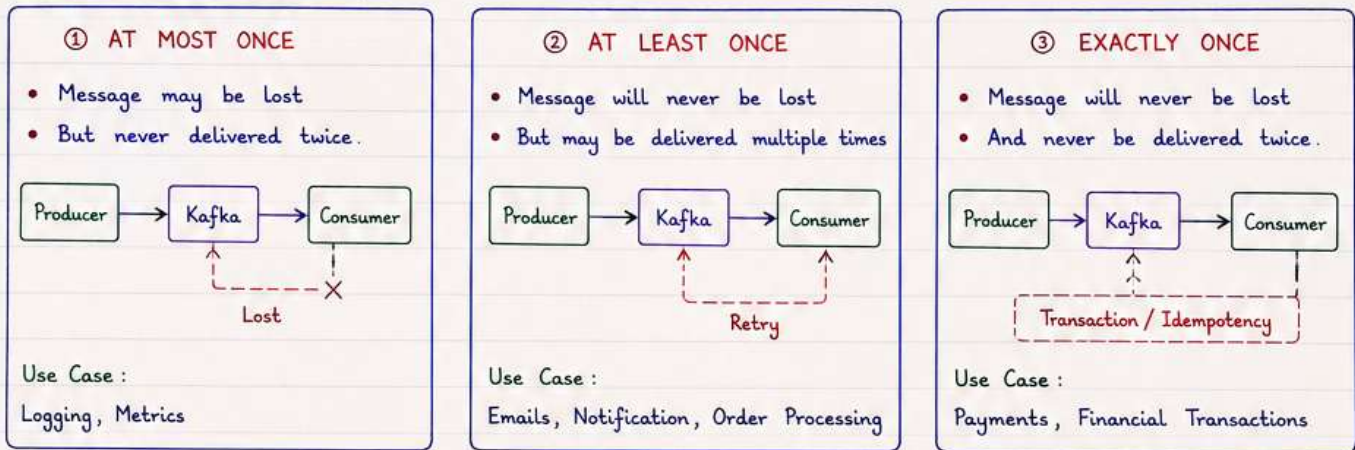
NOTE : Replication + ISR + Consumer Groups + Offsets together make Kafka highly available and fault tolerant.

More Replicas
= More Safety
= More Storage

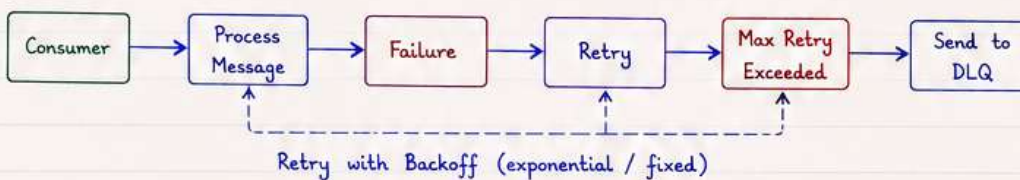
6. DELIVERY GUARANTEES, RETRY & DEAD LETTER QUEUE

Kafka provides different delivery guarantees depending on your use case.

6.1 DELIVERY GUARANTEES



6.2 RETRY MECHANISM (CONSUMER SIDE)



RETRY EXAMPLE FLOW

- ① Message consumed
- ② Processing fails
- ③ Retry 1 → fail
- ④ Retry 2 → fail
- ⑤ Retry N → fail
- ⑥ Send to DLQ

6.3 DEAD LETTER QUEUE (DLQ)

- Messages that fail after multiple retries are sent to a separate Topic (DLQ).
- Helps in manual analysis & reprocessing.



SPRING BOOT CONSUMER RETRY + DLQ (application.yml)

```
spring:
  kafka:
    consumer:
      bootstrap-servers: localhost:9092
      group-id: order-group
      auto-offset-reset: earliest
    listener:
      ack-mode: manual_immediate
      type: batch
    properties:
      spring.json.trusted.packages: "*"
      spring.deserialization.value.delegate.class:
        org.springframework.kafka.support.serializer.JsonDeserializer
```

RETRY & DLQ CONFIG (application.yml)

```
spring:
  kafka:
    listener:
      retry:
        enabled: true
        max-attempts: 3
        backoff:
          initial-interval: 2000
          multiplier: 2.0
          max-interval: 10000
      dead-letter-topic: orders.dlq
      dead-letter-republish: true
```

6.4 PRODUCER RELIABILITY SETTINGS

PROPERTY	VALUE	DESCRIPTION
acks	all	Leader + all ISR replicas must acknowledge.
retries	Integer.MAX_VALUE	Retry until successful.
enable.idempotence	true	Ensures exactly-once for producer.
max.in.flight.requests.per.connection	5 (default)	Use 1 for strict ordering with idempotence.

BEST PRACTICES

- ✓ Use meaningful keys.
- ✓ Choose right number of partitions.
- ✓ Enable retries with backoff.
- ✓ Use DLQ for better error handling.
- ✓ Monitor lag, consumer health.
- ✓ Make consumers idempotent for exactly-once.

★ QUICK SUMMARY

At Most Once → Fast, may lose data	Retry + Backoff → Handle temporary failures
At Least Once → Safe, may duplicate	DLQ → Handle permanent failures
Exactly Once → Safe + No duplicates	Replication → High availability & durability

Choose the right guarantee based on your business requirement!

7. KAFKA IN SPRING BOOT

Let's build a simple Spring Boot application with Kafka.

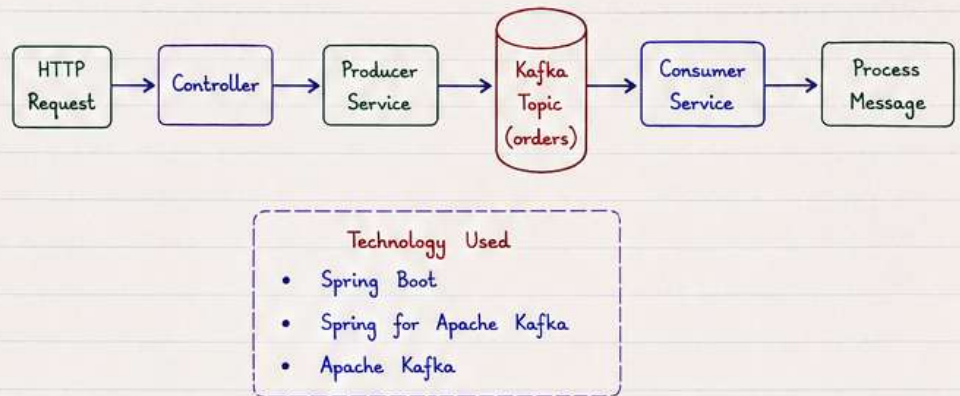
7.1 PROJECT STRUCTURE

```

src
├── main
│   └── java
│       ├── com.example.kafka
│       │   ├── controller
│       │   │   └── MessageController.java
│       │   ├── producer
│       │   │   └── KafkaProducerService.java
│       │   ├── consumer
│       │   │   └── KafkaConsumerService.java
│       │   └── config
│       │       └── KafkaConfig.java
│   └── resources
│       └── application.yml

```

7.2 FLOW DIAGRAM



7.3 PRODUCER EXAMPLE

① application.yml (Producer Config)

```

spring:
  kafka:
    bootstrap-servers: localhost:9092
    producer:
      key-serializer: org.apache.kafka.common.serialization.StringSerializer
      value-serializer: org.springframework.kafka.support.serializer.JsonSerializer
      properties:
        spring.json.add.type.headers: false

```

② Producer Service

```

@Service
public class KafkaProducerService {
    @Autowired
    private KafkaTemplate<String, OrderEvent> kafkaTemplate;

    public void sendOrder(OrderEvent order) {
        kafkaTemplate.send("orders", order.getId(), order);
    }
}

```

③ Controller

```

@RestController
@RequestMapping("/api/orders")
public class MessageController {
    @Autowired
    private KafkaProducerService producerService;

    @PostMapping
    public String createOrder(@RequestBody OrderEvent order) {
        producerService.sendOrder(order);
        return "Order sent to Kafka!";
    }
}

```

★ NOTE :

- We are sending OrderEvent object to Kafka Topic 'orders'.
- Key = orderId (helps in ordering in partition).
- Value = OrderEvent (our payload).

7.4 CONSUMER EXAMPLE

① application.yml (Consumer Config)

```

spring:
  kafka:
    bootstrap-servers: localhost:9092
    consumer:
      group-id: order-group
      key-deserializer: org.apache.kafka.common.serialization.StringDeserializer
      value-deserializer: org.springframework.kafka.support.serializer.JsonDeserializer
      properties:
        spring.json.trusted.packages: "*"
        auto-offset-reset: earliest
        enable-auto-commit: false

```

② Consumer Service

```

@Service
public class KafkaConsumerService {
    @KafkaListener(topics = "orders", groupId = "order-group")
    public void listen(OrderEvent order) {
        System.out.println("Received Order : " + order.getId());
        // Process the order
    }
}

```

@KafkaListener PARAMETERS

- topics : Kafka topic name
- groupId : Consumer group
- containerFactory : (Optional) custom factory
- autoStartup : (Optional) start listener automatically or not

7.5 ORDER EVENT (SAMPLE)

```

public class OrderEvent {
    private String orderId;
    private String userId;
    private double amount;
    // getters and setters
}

```

Flow Summary

1. User calls REST API
2. Controller → Producer
3. Producer sends to Kafka
4. Consumer receives message
5. Business Logic is executed

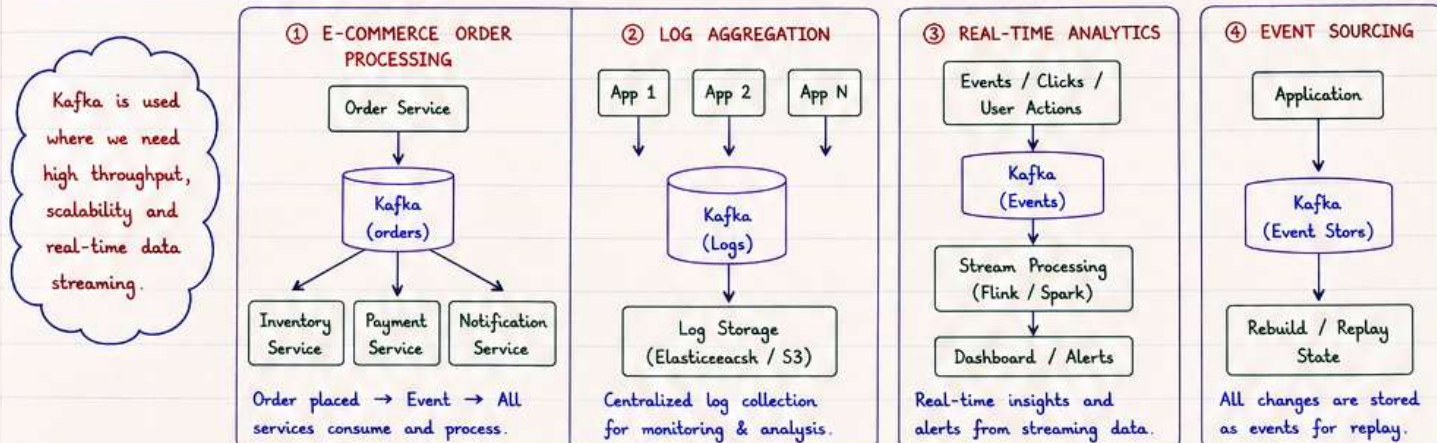
KEY TAKEAWAYS

- | | | |
|-------------------------------------|-----------------------------------|--------------------------------------|
| ✓ Easy integration with Spring Boot | ✓ Loose coupling between services | ✓ Reliable message delivery |
| ✓ Asynchronous communication | ✓ Highly scalable | ✓ Supports real-time event streaming |

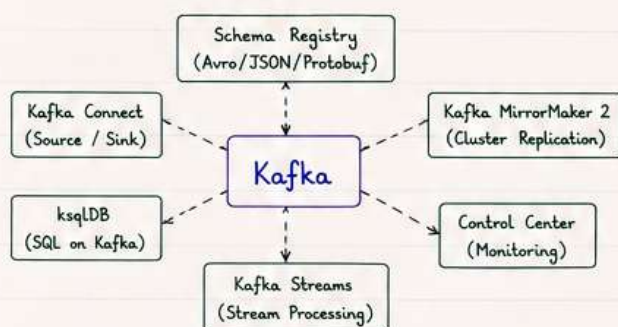
Spring Boot + Kafka
= Perfect combination for
modern microservices !

8. REAL WORLD USE CASES & BEST PRACTICES

8.1 REAL WORLD USE CASES



8.2 KAFKA ECOSYSTEM



8.3 MESSAGE FLOW SUMMARY



- Messages are immutable and appended to log.
- Consumers read at their own pace.
- Offsets track what has been consumed.
- Retention policy decides how long data is kept.

8.4 BEST PRACTICES

1. PRODUCER BEST PRACTICES

- ★ Use meaningful keys for better partitioning.
- ★ Enable acks=all for durability.
- ★ Enable retries with appropriate backoff.
- ★ Enable idempotence=true to avoid duplicates.
- ★ Use compression (snappy/lz4) for better throughput.

2. CONSUMER BEST PRACTICES

- ★ Use Consumer Groups for parallelism.
- ★ Commit offsets properly.
- ★ Handle failures with retry and DLQ.
- ★ Set max.poll.records and max.poll.interval.ms wisely.
- ★ Make consumers idempotent.

3. TOPIC & PARTITION BEST PRACTICES

- ★ Choose right number of partitions (not too less / not too many).
- ★ Use keys to keep related data in same partition (ordering).
- ★ Monitor partition skew.
- ★ Replication factor ≥ 3 for production.

4. OPERATIONAL BEST PRACTICES

- ★ Monitor broker, lag, and throughput.
- ★ Set proper retention policy.
- ★ Use separate topics for different use cases.
- ★ Backup critical data.
- ★ Regularly clean up old topics.

8.5 KAFKA vs RABBITMQ (QUICK COMPARISON)

FEATURE	KAFKA	RABBITMQ
Model	Event Streaming Platform	Traditional Message Queue
Data	Retained (Log)	Deleted after consumption.
Throughput	Very High (Millions msg/sec)	Lower compared to Kafka
Scalability	Horizontal (Partitions)	Limited (Queue based)
Use Case	Real-time streaming, analytics, logs, event sourcing	Task queues, RPC, simple messaging
Storage	Disk based (high durability)	Memory + Disk
Ordering	Per partition	Per queue

8.6 KAFKA CHEAT SHEET

- **Broker** → Kafka server.
- **Topic** → Logical channel.
- **Partition** → Split of topic for parallelism.
- **Offset** → Position of message in partition.
- **Leader** → Handles reads and writes.
- **Follower** → Replicate data from leader.
- **ISR** → In-Sync Replicas.
- **Consumer Group** → Group of consumers.
- **DLQ** → Dead Letter Queue.
- **Retention** → How long data is kept.
- **Replication** → Data copied across brokers.

★ FINAL NOTES

- ✓ Kafka is a Distributed, Fault Tolerant, Scalable Event Streaming Platform.
- ✓ It is not just a message queue, it is a log system.
- ✓ Design topics, partitions and consumer groups carefully.
- ✓ Monitor, test failure scenarios and handle retries.
- ✓ Keep it simple, reliable and observable.

Understand the flow,
Handle failures gracefully
And build systems that
scale with confidence!

REMEMBER

- ✓ Data never disappears in Kafka (until retention).
- ✓ Consumers read at their own speed.
- ✓ High throughput + High reliability.
- ✓ The heart of Modern Data Architecture!

★ Think in Events → Stream in Kafka → Act in Real-time ★

COMMENT PDF

— TO GET —

PDF NOTES



Aman Views

Java • SQL • System Design • DSA
Backend Development • Web Development
QA Testing

Handwritten Notes



TO GET LINK OF ALL OUR NOTES,
CHECK THE LINK IN BIO.